

When Does MPLS Traffic Engineering Need Sequential Reinforcement Learning?

Technical report

Uğur Efe Yiğit

Contents

1	Introduction	1
1.1	Introduction	1
2	Background and related work	3
2.1	Background and Related Work	3
3	Problem formulation	5
3.1	Problem Formulation	5
3.2	Environment details	6
4	Methods	8
4.1	Methods	8
4.2	Hyper-parameters and budgets	9
5	Experimental setup	11
5.1	Experimental Setup	11
6	Results	12
6.1	Results	12
6.1.1	Reproducing the closed study	12
6.1.2	RQ1: Does sequential PPO outperform contextual learning?	12
6.1.3	RQ4: How sequential is the decision problem?	13
6.2	Why the Bandit Wins	14
6.3	Additional results	15
7	Engineering, validation and reproduction	18
7.1	Governance and integrity checks	18
7.2	Action-mask audit	19
7.3	Equivalence and reproduction tests	19
8	Discussion and limitations	20
8.1	Limitations	20
9	Process record: failures and design history	21
9.1	From Environment V1 to V2	21
9.2	The closed V2 study	21
9.3	Failures in this study	21
10	Conclusion	23

Abstract

Abstract

TODO

Chapter 1

Introduction

1.1 Introduction

Traffic engineering (TE) decides which paths carry which traffic so that a backbone delivers its demand without congestion, SLA violations or needless reconfiguration. In MPLS networks the decision is concrete: each traffic trunk rides one label-switched path, and an operator or controller moves trunks between a few precomputed candidates as load changes [10, 11]. Because traffic varies over the day and failures arrive unannounced, reinforcement learning (RL) is an appealing tool, and deep RL has been applied to routing and TE in many forms [3, 14, 50, 54, 56].

RL earns its machinery—value functions, discounting, credit assignment over long horizons—only if today’s decision changes tomorrow’s options in ways that a myopic decision rule would get wrong. Whether that is true of TE is rarely measured. The strongest recent TE systems argue the opposite for the standard setting, in which routing is re-optimized from scratch every interval and therefore has no memory: Teal reduces its RL objective to a one-step return [53], and DOTE reports that direct optimization beats an RL-based scheme by a wide margin [44]. Operational MPLS-TE is different in exactly the property those arguments rely on. LSP moves are incremental and signalled, controllers limit how many trunks change at once, stability timers prevent a trunk from moving again too soon, and protected traffic may move only where capacity is guaranteed. The current assignment is then part of the state, a move constrains the future, and the decision problem is a genuine Markov decision process.

This paper asks, for such a formulation, *whether sequential RL is needed, and why or why not*. We study a flow-level MPLS backbone (18 routers, 64 directed links, 17 demands with SLAs, four candidate LSPs each) in which a controller may move one demand per five-minute interval, moves cost reconfiguration effort, a three-interval hold-down follows each move, and an action mask enforces failed links, hold-downs and protected-class capacity. We compare MaskablePPO [30, 48] with a masked neural contextual bandit that regresses only the immediate reward of each action, under identical observations, masks, budgets and paired traffic.

The comparison was first run as a governed study by the project that developed the environment, which found the bandit ahead. We treat that study as a claim to be tested. Our contributions are:

- **A reproduction and its lesson.** Retraining all of the closed study’s runs on different hardware with byte-identical training traffic, the bandit’s result reproduces and PPO’s does not: floating-point differences alone move PPO’s per-root holdout return by up to 41 points and the bandit’s by at most 4 (§6.1.1).
- **A controlled comparison with a non-learning optimizer.** On 5 training roots and 140 disjoint test episodes the bandit beats PPO on every root and is statistically indistinguishable from a per-interval

min-max-utilization MILP controller that does not learn at all (§6.1.2).

- **A mechanism.** PPO’s deficit is dominated by reconfiguration cost: on most roots it settles into flipping a demand at every hold-down expiry, a small immediate cost that its long-horizon advantage estimates fail to attribute (§6.2). Horizon and algorithm are separated by varying the discount within each learner family (§??).
- **A measured sequentiality diagnostic.** Clairvoyant rollouts show that the problem is far from myopic per decision under the true future, but that almost all of the non-myopic value is anticipation of exogenous traffic change; with traffic held fixed, the myopic choice is the 24-interval choice in 88 % of states (33 % under the true future) (§6.1.3).
- **A regime where sequential RL is necessary.** Delaying the effect of a move by one interval—a controlled form of actuation latency—removes the bandit’s signal by construction and tests whether the sequential learners recover the lost value at the same budget (§??).

We make no algorithmic or theoretical claim. All code, configurations, per-episode results and the scripts that produce every number, table and figure in this paper are part of the artifact.

Chapter 2

Background and related work

2.1 Background and Related Work

MPLS traffic engineering. MPLS-TE steers traffic trunks onto explicitly routed label-switched paths (LSPs) [10, 12]; RSVP-TE signals them and moves them with make-before-break [11], and the same “choose one of a few candidate paths” primitive applies to segment-routing policies [21]. Classical TE solves an optimization per traffic matrix—weight setting [22], oblivious and semi-oblivious routing [8, 34], robust TE [51], multicommodity-flow formulations minimizing the maximum link utilization [52]—or adapts online across a fixed set of paths [19, 33]. Production SDN WANs re-solve periodically [28, 31]. TeXCP [33] made the point that online TE must be stable as well as responsive, which is why our formulation prices moves and imposes a hold-down.

Learning for routing and TE. RL for routing dates to Q-routing [17]. Deep RL has been applied to intradomain routing [49, 50], utility-maximizing TE [54], critical-flow rerouting [43, 56], GNN-based and multi-agent TE [3, 13, 14, 25, 26, 29]. The strongest recent systems, however, are not sequential: Teal argues that a TE allocation for one interval does not affect future intervals and reduces the RL return to one step [53]; DOTE trains a network end to end on the realized utilization and reports that an RL-based scheme is markedly worse [44]; FIGRET and HARP are one-shot learned TE [4, 38]. Several DRL-TE formulations are, in our reading, contextual bandits in disguise: the state is the exogenous traffic history and actions do not influence it [50, 56]. Our setting differs in the property those arguments rely on: the routing configuration persists, changes are incremental (one LSP per interval), costly and subject to hold-down, so the current assignment is endogenous state.

When simple methods win. Deep-RL results are sensitive to implementation and tuning and need careful statistics [2, 7, 20, 27, 42]; random search matches model-free deep RL on standard control tasks [39]; in networking, a year-long randomized trial found that learned bitrate control did not beat a simple buffer-based rule [55], and a re-examination of RL for optical resource allocation found that improved heuristics match or beat the published agents [18]. Our contribution is not another instance of this pattern but a controlled attribution of it to horizon versus algorithm.

Contextual bandits and horizon. A regression oracle on immediate rewards with simple exploration is a standard contextual-bandit design [23, 37], and greedy methods are competitive when contexts are diverse [15]. Theory shows that long horizons need not make RL much harder than bandits in sample complexity [57], while a planning horizon or discount shorter than the true one can improve learned policies with limited data [6, 32, 45]. Laidlaw et al. define the *effective horizon*, the lookahead depth needed to find

the optimal action when leaves are evaluated by a base policy, and show it predicts deep-RL success [35, 36]. Congested bandits formalize rewards that depend on recent actions over a short window [9]. Potential-based shaping [41] is policy-invariant only for the discount it is defined with—a caveat for any $\gamma = 0$ learner trained on a shaped reward. To our knowledge no TE study measures the sequentiality of its decision problem or separates horizon from algorithm in a learner comparison.

Action masking. Masking invalid actions in policy-gradient methods is theoretically justified and practically important when many actions are illegal [30]. Masks are a form of safe exploration with external knowledge [24]; constrained MDPs [1, 5] are the alternative we do not pursue. Flow-level simulation may not transfer to packet dynamics [16]; we return to this in Section 8.1.

Chapter 3

Problem formulation

3.1 Problem Formulation

We state the decision problem exactly as implemented (Environment V2 of the artifact, frozen at commit dca533b); Appendix 3.2 gives every constant.

Network and demands. The backbone is a directed graph $G = (V, E)$ with $|V| = 18$ routers (four ingress and four egress provider-edge routers, eight core and two aggregation routers) and $|E| = 64$ directed links (32 bidirectional links, 250 Mb/s to 2,000 Mb/s per direction; Fig. ??). There are $|\mathcal{D}| = 17$ demands in six classes, each with a priority $\pi_d \in \{1, \dots, 6\}$, a delay and a loss SLA, and a *protected* flag (voice and critical traffic). Each demand has a fixed, ordered set of $k = 4$ candidate label-switched paths $\mathcal{P}_d = (p_{d,0}, \dots, p_{d,3})$, computed once by Yen’s algorithm under role, hop-count and propagation-delay constraints; $p_{d,0}$ is the administratively shortest valid path.

Exogenous traffic. The offered rate of demand d is $v_d(t) = b_d f_{\kappa(d)}(h(t)) m \eta_d(t) \text{clip}(1 + n_d(t), 0.4, 2)$: a class-specific diurnal profile, a scenario multiplier, scripted or randomized burst / flash-crowd factors and per-demand AR(1) noise ($n_d(t) = 0.9 n_d(t-1) + \sigma_d \varepsilon$, one step per simulated minute). Link failures and recoveries are scripted or randomized events. Nothing in the exogenous process ξ_t depends on routing, which makes comparisons between controllers exactly paired.

State and transitions. The controllable state y_t holds, for each demand, its current candidate x_d , a disconnection flag, the remaining *dwell* (hold-down) ω_d , the previous traffic-engineering (TE) path and its age. The Markov state is $s_t = (y_t, \xi_t)$. Decisions are taken every 5 min; an interval is simulated as five one-minute ticks in which a damped fixed-point solver computes carried flow, per-link loss (0 below 90 % utilization, quadratic to 2 % at 100 %, $1 - 0.98/\rho$ above), M/M/1-shaped queueing delay and per-demand SLA status. A link failure triggers immediate, free fast-reroute of the affected demands to their cheapest live candidate. Because ξ is action-independent,

$$P(s_{t+1} | s_t, a_t) = P_\xi(\xi_{t+1} | \xi_t) \mathbb{K}[y_{t+1} = \Psi(y_t, a_t, \xi_{t:t+1})], \quad (3.1)$$

so an action influences the future *only* through the routing state, which persists until changed.

Actions and masks. $\mathcal{A} = \{0\} \cup \{(d, p)\}$, $|\mathcal{A}| = 69$: action 0 changes nothing; (d, p) moves demand d to candidate p . At most one TE change is made per interval. The legality mask is

$$m_t(d, p) = \mathbb{K}[p_{d,p} \text{ is up}] \mathbb{K}[p \neq x_d] \mathbb{K}[\omega_d = 0] \mathbb{K}\left[\neg \text{prot}_d \vee \max_{e \in p_{d,p}} \hat{G}_e / c_e \leq 1\right], \quad (3.2)$$

with $m_t(0) = 1$, where $\hat{G}_e$ is the projected gross load on link e after the move. An accepted move starts a three-interval dwell. The last factor couples demands: where one demand is routed changes which moves are legal for protected demands.

Reward. With DR the delivered ratio, PD/UD the priority-weighted protected and unprotected disconnected fractions, SEV a bounded priority-weighted SLA severity, $\rho^{\max}$ the worst link utilization and OV the overload ratio,

$$r_t = \underbrace{2\text{DR} - 30\text{PD} - 8\text{UD} - 6\text{SEV} - 2g(\rho^{\max}) - 6\text{OV}}_{U_t} + 0.2(0.995\Phi_{t+1} - \Phi_t) - C_t^{\text{TE}} - 0.05\mathbb{I}[\text{rejected}], \quad (3.3)$$

where $g(\rho) = \log(1 + \max(0, (\rho - 0.7)/0.3))$, $\Phi = \tanh(U/10)$ evaluated at interval boundaries, and the move cost $C^{\text{TE}} = 0.08 + 0.30\text{vol} + 0.12\text{div} + 0.30\text{rev}$ charges the moved traffic share, the edge-set divergence between old and new path and returns to the previous path within six intervals. The reward is the exact sum of twelve logged components.

What kind of problem is this? With respect to s_t the problem is a finite-horizon MDP with state-dependent action sets: routes persist, moves lock a demand for three intervals, reversal costs depend on history, and the protected-class mask couples demands. The learners observe $o_t \in [0, 1]^{604}$, which contains the routing state (current and previous path, dwell, path age), per-link utilization, per-demand health relative to SLA, and the projected bottleneck of every candidate move, but not the clock, the noise state or future events; with respect to ξ the problem is therefore partially observable. A contextual bandit that regresses $\mathbb{E}[r_t \mid o_t, a_t]$ discards the persistence of a new configuration beyond the first interval, the opportunity cost of the dwell lock, future reversal costs and the effect on other demands' legal sets. It keeps the first interval of a move's effect because, in the frozen environment, a move carries traffic in the interval in which it is made. Section 6.1.3 measures how much the discarded terms matter.

3.2 Environment details

All values are generated from the configuration files by `scripts/study/analyze.py`. The topology has 18 routers, 32 bidirectional links (64 directed), 45.0 Gb/s of total directed capacity, and 17 demands whose base (profile-peak) rates sum to 2.49 Gb/s.

Table 3.1: Traffic classes. Priority weights $q_d = \pi_d/6$ enter the SLA and disconnection terms; protected classes may only be moved onto candidates whose projected gross bottleneck utilization is at most 100 %.

Class	Priority	Delay SLA (ms)	Loss SLA (%)	Protected	Profile	Demands
voice	5	60	0.5	yes	business	3
video	4	120	1	no	evening_video	4
vpn	3	150	1	no	business	3
besteffort	2	250	3	no	consumer	3
bulk	1	400	5	no	night_bulk	3
critical	6	80	0.2	yes	flat	1

Observation (604 features, feature-major, all in $[0, 1]$). Per directed link: $\text{sat}(\rho_e) = \rho_e/(1 + \rho_e)$ and the up flag (2×64). Per demand (28×17): $\text{sat}(\text{offered}/\text{base})$, priority/6, protected flag, $\text{sat}(\text{delay}/\text{SLA})$, $\text{sat}(\text{loss}/\text{SLA})$, path age/12 (capped), dwell/3, disconnected flag, one-hot current path (4), one-hot previous

Table 3.2: Scenarios. Learners train only on `random_day`, whose bursts, single link failure (probability 0.5) and flash crowd (probability 0.3) are drawn per episode; all seven evaluation scenarios are scripted and differ from the training distribution.

Scenario	Start (h)	Decisions	Multiplier	Noise σ	Events
<code>full_day</code>	0	288	1	1	none
<code>evening_peak</code>	16	84	1.15	1	none
<code>flash_crowd</code>	18	60	1.1	1	flash crowd
<code>link_failure</code>	17	60	1.1	1	link down, link up
<code>deceptive_local_optimum</code>	9	60	1	0.8	burst
<code>ood_double_failure</code>	10	60	1.1	1	link down, link up
<code>overload_stress</code>	18	48	1.6	1	none
<code>random_day</code>	0	288	1	1	randomized

TE path (4), candidate liveness (4), candidate propagation delay/100 ms (4), and sat of the projected gross bottleneck utilization of moving to each candidate (4). Time of day, episode progress and future events are deliberately excluded.

Episode boundaries. Events scheduled at minute t are applied in the right-closed window $(t - 1, t]$, so the observation at a boundary reflects every event up to that time. Episodes never terminate; they are truncated at the scenario end, so paired controllers have identical horizons.

Chapter 4

Methods

4.1 Methods

All learners see the same observation and the same authoritative mask, train on the same randomized scenario for the same number of environment transitions, and are evaluated deterministically (masked argmax).

MaskablePPO. We use the `sb3-contrib` implementation [46] with invalid-action masking [30]: logits of illegal actions are set to -10^8 , so $\pi_\theta(a | o, m) \propto m(a)e^{z_\theta(o)_a}$, and the same mask is applied in rollouts, gradient steps and inference. Advantages use GAE [47] and the clipped surrogate objective [48] with value and entropy terms. The study’s default configuration (inherited unchanged from the closed study) is $\gamma = 0.995$, $\lambda = 0.95$, learning rate $3 \cdot 10^{-4}$, 16 environments $\times$ 512 steps per rollout, batch 512, 8 epochs, clip 0.2, entropy 0.01, separate [256, 256] tanh networks.

Masked neural contextual bandit. A 604–256–256–69 ReLU network $\hat{r}_\theta(o, \cdot)$ predicts the immediate reward of every action. Actions are chosen by masked ε -greedy (ε linear from 0.2 to 0.02 over $2 \cdot 10^5$ transitions; random actions are drawn from the legal set). A replay buffer of the last 10^5 tuples (o, a, m, r) is sampled every 64 transitions for one Adam step on the Huber loss between the *selected* head and the observed reward. There is no next state, discount, target network or bootstrapping. At 400k transitions the bandit performs 6,187 gradient steps, PPO 6,144.

Masked Q-learner with discount γ . To vary the planning horizon *without changing anything else*, we extend the bandit by a single parameter. For $\gamma > 0$ the regression target becomes the normalized double-DQN [40] target

$$y = (1 - \gamma)r + \gamma Q_{\hat{\theta}}(o', \arg \max_{a': m'(a')=1} Q_\theta(o', a')), \quad (4.1)$$

with a target network copied every 250 updates; the maximization is over legal successor actions only. The factor $(1 - \gamma)$ rescales all action values of one γ by a common constant and leaves the greedy policy unchanged; it keeps the Huber transition point comparable across γ . With $\gamma = 0$ the learner is the bandit, bit for bit (unit-tested).

Non-learning controllers. *Static* keeps every demand on $p_{d,0}$. *Greedy* moves the largest demand on the hottest link (if above 85 %) to the candidate with the lowest bottleneck utilization (margin 5 %). *CSPF* re-replaces demands every six intervals in priority order onto the cheapest candidate with 10 % reservation headroom (hysteresis 8 %). We add *MILP-track*: every interval it solves the classical min-max-utilization path-selection MILP over the four candidates per demand (HiGHS via SciPy), with a small penalty for

Algorithm 1 Masked value learner (bandit for $\gamma = 0$). Source: `experiments/masked_bandit.py`, `study/learners.py`.

```

1: observe  $o$  from 16 environments
2: for each vector step do
3:    $m \leftarrow \text{masks}; a \leftarrow \arg \max_{a:m(a)=1} Q_\theta(o, a)$ , or a uniform legal action w.p.  $\epsilon$ 
4:    $o', r \leftarrow \text{step}(a)$ ; store  $(o, a, m, r, [o', m'])$ 
5:   if every 4th vector step and  $\geq 4096$  stored then
6:     sample 512 tuples;  $y \leftarrow r$  if  $\gamma = 0$  else Eq. (4.1)
7:     Adam step on  $\text{Huber}(Q_\theta(o, a) - y)$  (selected heads only)
8:    $o \leftarrow o'$ 

```

changing a demand’s path, and, if the optimum lowers the maximum utilization by at least 0.02, submits the single legal move towards it with the largest volume. Because V2 allows one change per interval, only the first legal proposal of any controller is applied.

Clairvoyant lookahead references. *Oracle-H* clones the simulator, including the traffic generator, and scores each legal first action by the H -interval return of taking it and then making no further change; it plays the best (ties: no-op). *Oracle-1* maximizes the exact immediate reward, the quantity the bandit estimates; larger H adds clairvoyant lookahead. These are references that see the future, not deployable controllers and not optimal policies.

Controlled delayed effect. To create a regime in which sequential credit assignment is unavoidable, we add a variant in which a move requested at interval t is charged at t but takes effect at $t + L$ (actuation latency, or a periodic commit cycle). Pending requests lock their demand and are visible in the observation (689 features); a request that has become illegal by $t + L$ is cancelled. With $L = 0$ the variant is bit-identical to the frozen environment (unit-tested). With $L \geq 1$ the immediate reward of a move contains its cost but none of its effect, so a bandit cannot credit it.

4.2 Hyper-parameters and budgets

Table 4.1: Learner configurations. All values are read from `configs/training.yaml` (PPO) and `configs/experiments/learning_v2.yaml` (bandit/Q); the resolved configuration of every run is stored in its manifest.

	MaskablePPO	Masked bandit / Q-learner
Network	separate [256, 256] tanh (policy, value)	604–256–256–69 ReLU
Optimizer	Adam, $3 \cdot 10^{-4}$, grad-norm clip 0.5	Adam, $3 \cdot 10^{-4}$, grad-norm clip 1.0
Discount	$\gamma = 0.995$, GAE $\lambda = 0.95$	$\gamma = 0$ (bandit); $\{0.5, 0.9, 0.99\}$ (Q)
Data per update	rollout $16 \times 512 = 8192$, 8 epochs, batch 512	replay 10^5 , batch 512, every 64 transitions
Gradient steps / 400k	6,144	6,187
Exploration	policy entropy, coef. 0.01	masked ϵ -greedy, $0.2 \rightarrow 0.02$ over $2 \cdot 10^5$
Loss	clipped surrogate (0.2) + $0.5 \times$ value MSE	Huber on selected head
Target	GAE returns	r (bandit); Eq. (4.1), target copy every 250 updates (Q)
Masking	logits $\rightarrow -10^8$	scores $\rightarrow -\infty$; random actions from legal set
Warm-up	–	4,096 transitions

PPO sensitivity grid (E3). One-factor-at-a-time around the default, on training root 42, selected on validation seeds only: learning rate $\{10^{-4}, 10^{-3}\}$; rollout length per environment $\{128 \text{ (batch 256)}, 2048 \text{ (batch 1024)}\}$; entropy $\{0, 0.03\}$; clip 0.1; network $[64, 64]$; and $\gamma \in \{0, 0.9\}$ (E2).

Chapter 5

Experimental setup

5.1 Experimental Setup

Training. Every learner trains on `random_day` (24 h, 288 decisions, randomized bursts, failures and flash crowds per episode) for 400,000 transitions from 16 vectorized environments, with checkpoints every 50,000 transitions. The two learners of a training root share the training episode-seed ledger byte for byte, so roots are paired.

Evaluation protocol. Seven scripted scenarios (Table 3.2) are evaluated with three disjoint seed sets that are fixed in code before any new result was produced: *validation* seeds (2001–2005, 35 episodes) select a checkpoint (highest mean return; ties to the earlier); *test* seeds (3001–3020, 140 episodes) are used only for reporting, for the selected and for the final checkpoint; and *diagnostic* seeds (4001–4003) are used only for structural diagnostics. The closed study’s selection seeds (101–105) and final-holdout seeds (1001–1005) are used only to reproduce its numbers (Section 6.1.1). Evaluation is deterministic (masked argmax). Because traffic is exogenous, every policy faces byte-identical traffic on a given (scenario, seed), so all comparisons are paired.

Metric. The primary metric is the undiscounted operational return $R = \sum_t r_t$ of an episode, averaged over the 7 scenarios $\times$ 20 seeds. Scenarios have 48–288 decisions, so the average weights long scenarios more; we therefore also report per-scenario results and the operational components (delivered ratio, SLA-violating demand-intervals, maximum utilization, reroutes per hour, moved bandwidth).

Statistics. We separate two sources of uncertainty. For a *fixed* policy, 95 % intervals are scenario-stratified bootstrap intervals over test episodes (10,000 resamples). For a *learner*, the experimental unit is the training root: intervals are two-stage bootstrap intervals (resample roots, then scenario-stratified episodes within roots), and we also report a *t*-interval over root-level means and the number of roots on which a difference has a given sign. Paired differences are reported in reward units together with the paired standardized effect d_z . We do not use significance tests as decision rules; with five roots, root-level intervals are wide, and we say so where it matters [2, 42].

Hardware. All new results were produced on a 4-core Intel Xeon (2.1 GHz) virtual machine without GPU, one thread per run (PyTorch 2.14 CPU, SB3/SB3-contrib 2.9.0). The closed study trained on an RTX 4070 laptop GPU; its checkpoints were not archived, so we retrained them (Section 6.1.1).

Chapter 6

Results

6.1 Results

6.1.1 Reproducing the closed study

The study we build on reported, on its untouched final holdout (7 scenarios $\times$ seeds 1001–1005, three training roots), a mean return of 18.2 for the bandit and 9.0 for PPO, and a small PPO advantage in the deceptive-local-optimum scenario. Its checkpoints were not archived, so we retrained all six runs with the governed trainer at the same commit lineage on a CPU-only machine (the originals used CUDA) and repeated its selection and holdout procedure exactly.

Three properties reproduce *exactly*: the 21 baseline (policy, scenario) holdout means and standard deviations (to $3 \cdot 10^{-14}$), the training episode-seed ledger of every root (byte-identical), and all integrity counters. The bandit reproduces statistically (18.0 vs. 18.2; Table 6.1); PPO does not (-12.0 vs. 9.0). With identical training traffic, the only difference between the executions is floating-point arithmetic, and it moved PPO’s per-root holdout return by up to 41 points while moving the bandit’s by at most 4: on 2 of 3 roots the reproduced PPO never reached a positive validation return at any checkpoint. The bandit’s advantage reappeared on all three roots and grew to 30.1 (two-stage bootstrap 95 % CI [4.4, 50.8]); the historical deceptive-scenario PPO advantage (-1.1 for the bandit) did not reproduce (bandit – PPO = 14.7 [10.6, 18.9]).

We draw two methodological conclusions before any new experiment. First, PPO’s outcome in this environment has a large run-to-run variance that is not captured by three roots, so the remainder of the paper uses five roots for the primary comparison and reports root-level uncertainty. Second, the bandit’s result is robust to the perturbation; this is the first indication that its advantage is not an accident of one execution.

6.1.2 RQ1: Does sequential PPO outperform contextual learning?

Table 6.2 reports all policies on the 140 test episodes. The bandit reaches a mean return of 25.2 [21.5, 28.1] over 5 training roots and PPO 0.3 [-13.7, 13.6] over 5. On paired episodes the bandit’s advantage is 24.9 (two-stage bootstrap [12.2, 39.1]; t -interval over roots [3.4, 46.4]); it is positive on 5 of 5 roots and in 86 % of paired episodes ($d_z = 0.75$). The ordering does not depend on checkpoint selection: with the final 400k checkpoints the gap is 35.7 [17.2, 54.5] (5 of 5 roots). Per scenario (Table 6.3), the bandit is ahead in every scenario except the double-failure scenario, where the two are close; the largest gap is on the 24-hour `full_day` scenario. Both learners dominate the static, random and CSPF baselines; the bandit also beats greedy on every root, whereas PPO beats greedy on only some.

The most informative comparison is with a controller that does not learn at all. MILP-track, which solves a min-max-utilization path-selection problem on current telemetry every interval and moves one demand towards the solution, reaches 24.0 [21.9, 26.3]. The bandit and MILP-track are statistically indistinguishable

Table 6.1: Closed-study reproduction under its own protocol (selection on seeds 101–105, holdout 1001–1005, 35 episodes). Selected checkpoint and mean holdout return, historical (CUDA) vs. reproduced (CPU).

Learner	Root	Selected (hist./repr.)	Hist.	Repr.
Bandit	42	250k / 400k	16.1	14.7
Bandit	271828	400k / 350k	18.0	22.2
Bandit	314159	300k / 250k	20.5	17.2
PPO	42	250k / 350k	6.6	-20.1
PPO	271828	150k / 400k	12.2	-29.0
PPO	314159	350k / 200k	8.3	13.0

Table 6.2: Test results (7 scenarios $\times$ 20 seeds, 140 paired episodes). Learners: mean over training roots of the selected checkpoint, two-stage bootstrap 95 % CI; fixed policies: scenario-stratified bootstrap CI. Per-interval = return divided by the number of decisions, averaged over episodes. SLA = SLA-violating demand-intervals per episode. † clairvoyant reference (sees future traffic), not a controller.

Policy	Return [95 % CI]	Per-interval	Delivered	SLA	Mean max-util	Reroutes/h	Moved Mb/s
Masked bandit	25.2 [21.5, 28.1]	-0.302	0.9531	164	1.033	2.30	1794
MaskablePPO	0.3 [-13.7, 13.6]	-0.577	0.9479	190	1.047	6.38	4988
No-op	-95.5 [-98.0, -92.9]	-1.602	0.9058	352	1.385	0.00	0
Static SP	-96.5 [-98.9, -94.0]	-1.618	0.9040	346	1.396	0.37	204
Random valid	-107.5 [-111.3, -103.6]	-1.744	0.8944	401	1.315	11.66	9908
CSPF	-25.6 [-28.0, -23.2]	-0.896	0.9364	243	1.195	0.61	636
Greedy	4.0 [1.7, 6.3]	-0.484	0.9485	187	1.066	4.60	9293
MILP-track	24.0 [21.9, 26.3]	-0.295	0.9551	162	1.002	5.83	6312
Oracle-1 [†]	46.9 [45.2, 48.7]	0.010	0.9598	132	0.962	3.36	3233

(bandit – MILP-track = 1.2 [-2.5, 4.0]); PPO falls short of MILP-track (–23.8 [-37.6, -10.3]). Operationally, the bandit delivers 95.31 % of offered traffic with 2.30 reroutes per hour, against 94.79 % and 6.38 for PPO and 95.51 % and 5.83 for MILP-track: the learned myopic policy achieves the optimizer’s quality with less than half its churn.

6.1.3 RQ4: How sequential is the decision problem?

Formally the problem is an MDP (§3.1); the question is how much the omitted future terms matter. On 390 states visited by the greedy controller (diagnostic seeds 4001–4003, every fourth decision), we roll the cloned simulator forward from every legal action for up to 24 intervals—the action, then no further change—and compare the action that is best for the first interval with the action that is best over H intervals (Fig. 6.1). This is the effective-horizon idea of Laidlaw et al. [35] with a no-change base policy.

Under the true future, the problem is far from myopic per decision: the first-interval choice is the 24-interval choice in only 33 % of states, it captures 52 % of the available 24-interval gain, and in 59 % of states the 24-interval best move has a *negative* immediate advantage. *Holding traffic and link state at their current values* during the same rollouts, the picture reverses: agreement is 88 %, the myopic choice captures 97 % of the gain, and such “sacrifice” moves occur in 8 % of states. The pattern holds in every scenario (Appendix 6.3). Almost all of the non-myopic value is therefore *anticipation of exogenous change* (traffic growth, bursts, failures and recoveries), not the trade-off between a one-off move cost and a persistent gain. That value is visible to a learner only to the extent that future traffic is predictable from the observation, which contains current volumes but not the clock.

Table 6.3: Bandit — PPO paired return difference per scenario (selected checkpoints, all roots pooled; bootstrap 95 % CI over paired episodes) and share of paired episodes the bandit wins.

Scenario	Difference [95 % CI]	Bandit wins
Deceptive	9.1 [7.2, 11.0]	81 %
Evening peak	33.0 [29.5, 36.6]	100 %
Flash crowd	21.0 [17.9, 24.1]	89 %
Full day	69.5 [58.2, 81.3]	97 %
Link failure	18.4 [15.4, 21.4]	87 %
Double failure	0.6 [-2.2, 3.3]	57 %
Overload	22.8 [19.6, 25.7]	91 %

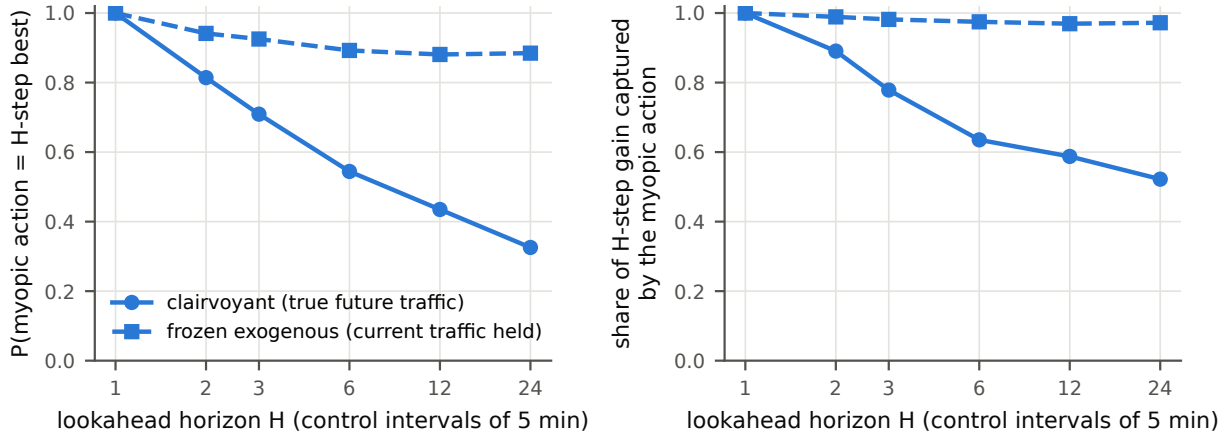


Figure 6.1: Sequentiality diagnostic on 390 paired states. Left: probability that the action with the best first-interval reward is also the best over H intervals (“take it, then change nothing”). Right: share of the best H -interval gain that the myopic action captures. Solid: rollouts carry the episode’s true future traffic and failures (clairvoyant). Dashed: traffic and link state held at their current values.

6.2 Why the Bandit Wins

Where the return goes. Splitting each episode’s return into network utility (delivery, disconnection, SLA, utilization and overload terms), shaping, and move costs (Table 6.4) localizes PPO’s deficit. Averaged over roots, the bandit earns a utility of 28.0 and pays -2.8 in move and reversal costs; PPO earns 18.4 and pays -18.1. PPO’s utility is not uniformly worse—its best root reaches 32.2, above the bandit’s best (31.6)—but on 3 of 5 roots more than half of its accepted moves are *reversals* (up to 80 %), and its per-root move costs range from -36.5 to -1.9, against -3.2 to -2.6 for the bandit. The shaping term contributes at most 0.02 per episode for any policy, so the bandit’s use of a shaped reward does not explain its advantage.

A degenerate oscillation. Figure 6.2 shows one test episode of the deceptive-local-optimum scenario. After moving at nearly every opportunity in the first hour, PPO settles into flipping one demand between two candidates every three intervals—exactly when the hold-down expires—and pays the reversal price each time. The bandit makes a handful of moves at the burst onset and otherwise holds its configuration. The behaviour is not an artefact of checkpoint selection: 2 of 5 final (400k) PPO checkpoints also reverse more than half of their moves.

Table 6.4: Return decomposition on the test episodes (means over episodes and, for learners, over roots). Utility = delivery + disconnection + SLA + utilization + overload terms; Costs = move and reversal terms.

Policy	Return	Utility	Costs	Shaping	Moves	Reversals
Masked bandit	25.2	28.0	-2.8	0.01	12.4	1.8
MaskablePPO	0.3	18.4	-18.1	0.01	46.9	32.6
MILP-track	24.0	34.3	-10.2	0.01	39.6	8.8
Greedy	4.0	14.4	-10.4	0.00	31.8	11.8
CSPF	-25.6	-24.9	-0.7	0.00	3.6	0.1
Static SP	-96.5	-96.1	-0.3	-0.01	1.8	0.0
No-op	-95.5	-95.5	0.0	-0.01	0.0	0.0
Oracle-1 [†]	46.9	50.6	-3.7	0.01	20.0	0.4

Table 6.5: What the learners’ actions are worth, from exact simulator rollouts on states each learner visits (test seeds 3001–3003, every third step). Top-1: share of states in which the learner’s action is the exact one-interval optimum; regret: exact one-interval reward lost; no-op: share of states in which the learner / the optimum chooses no-op; ΔG_6 : six-interval value over no-op (no-op continuation) of the learner’s action and of the exact myopic-best action.

	Root	Top-1	Regret	No-op (learner/opt.)	ΔG_6 learner	ΔG_6 myopic
Bandit	42	47 %	0.27	85 / 49 %	+0.24	+1.31
Bandit	271828	57 %	0.19	83 / 61 %	+0.20	+0.80
Bandit	314159	60 %	0.18	85 / 61 %	+0.27	+0.82
PPO	42	9 %	0.63	12 / 53 %	-0.08	+1.37
PPO	271828	7 %	0.67	5 / 49 %	-0.04	+1.57
PPO	314159	6 %	0.54	8 / 67 %	-0.25	+0.66

Credit assignment for small immediate costs. A reversal costs about 0.5 reward units once, while PPO’s advantage estimates aggregate, through GAE with $\gamma\lambda \approx 0.945$, the fluctuations of a reward that ranges from about +2 per interval in a healthy network to about −30 during protected outages (Eq. 3.3), driven by the training scenario’s random bursts and failures. A per-step cost of that size is small relative to the variance of the advantage target, and the policy gradient fails to attribute it to the step that incurs it. The bandit’s regression target *is* that step’s reward, so the cost enters its estimate with no noise from future intervals. Section ?? tests this explanation by removing PPO’s horizon.

What the learners’ actions are worth. On states visited by each learner on test episodes we compute, by cloning the simulator, the exact one-interval reward of every legal action and the six-interval value of the learner’s action, of the exact myopic-best action, and of no-op (Table 6.5). Neither learner is close to the exact myopic optimum, but they fail in opposite directions: the bandit is conservative (it chooses no-op far more often than the optimum does), PPO over-acts, and over six intervals PPO’s actions are on average worth less than doing nothing on some roots. PPO’s frequent moves therefore do not buy delayed value that a myopic criterion would miss.

6.3 Additional results

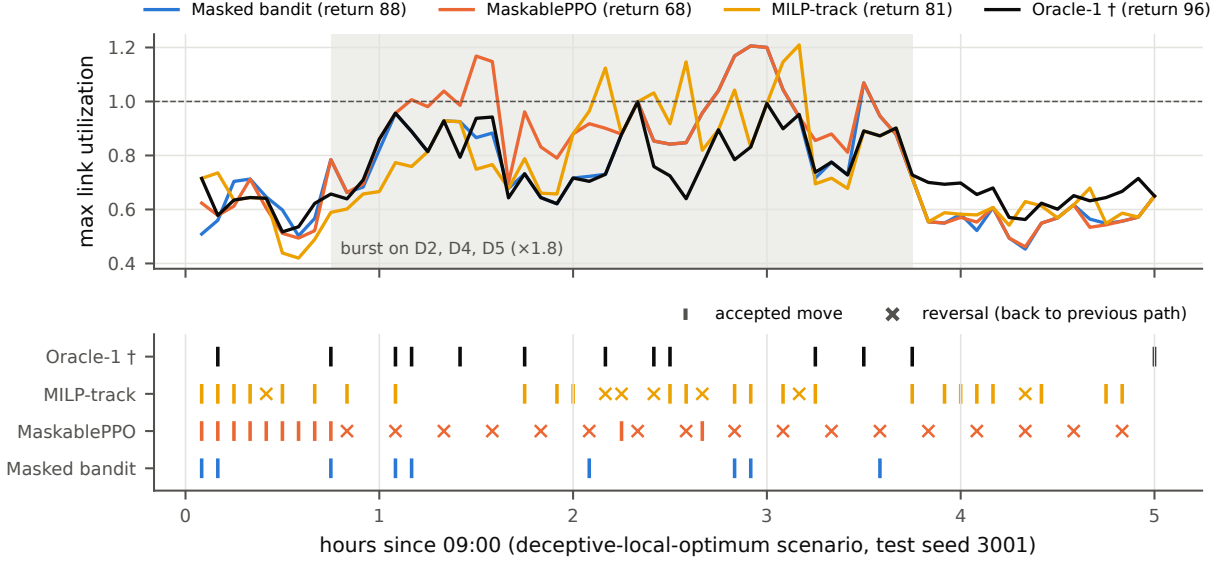


Figure 6.2: One test episode (deceptive-local-optimum, seed 3001; learners of root 314159). Top: worst link utilization; bottom: accepted moves ($\times$ = return to the previous path within six intervals). PPO flips one demand every three intervals, the hold-down period; the bandit makes few moves. Oracle-1 sees the next interval’s traffic and is shown for reference.

Table 6.6: Primary comparison per training root (mean test return over 140 episodes): selected checkpoint (validation seeds) and final 400k checkpoint.

Root	Masked bandit			MaskablePPO		
	Selected	Return	Final	Selected	Return	Final
42	350k	18.3	20.3	350k	-13.7	-47.4
141421	350k	25.0	18.6	300k	14.8	-9.4
161803	400k	28.5	28.5	300k	5.8	-4.6
271828	350k	28.4	24.1	400k	-22.5	-22.5
314159	250k	25.8	16.4	200k	16.9	13.2

Table 6.7: Learner minus reference, paired test returns (two-stage bootstrap 95 % CI over roots and episodes) and the number of roots on which the difference is positive.

Learner	Reference	Difference [95 % CI]	Roots > 0
Masked bandit	No-op	120.7 [117.0, 123.6]	5/5
Masked bandit	Static SP	121.7 [118.0, 124.6]	5/5
Masked bandit	Random valid	132.7 [128.7, 136.1]	5/5
Masked bandit	CSPF	50.8 [47.1, 53.7]	5/5
Masked bandit	Greedy	21.2 [17.6, 24.1]	5/5
Masked bandit	MILP-track	1.2 [-2.5, 4.0]	4/5
Masked bandit	Oracle-1 [†]	-21.7 [-25.4, -18.9]	0/5
MaskablePPO	No-op	95.7 [81.9, 109.2]	5/5
MaskablePPO	Static SP	96.7 [82.9, 110.2]	5/5
MaskablePPO	Random valid	107.8 [93.8, 121.2]	5/5
MaskablePPO	CSPF	25.9 [12.0, 39.4]	5/5
MaskablePPO	Greedy	-3.7 [-17.6, 9.8]	3/5
MaskablePPO	MILP-track	-23.8 [-37.6, -10.3]	0/5
MaskablePPO	Oracle-1 [†]	-46.6 [-60.3, -33.2]	0/5

Table 6.8: Sequentiality diagnostic per scenario at $H = 24$ (states with at least 24 remaining intervals): agreement of the myopic and the 24-interval best action, share of the 24-interval gain captured by the myopic action, and share of states in which the 24-interval best action has negative immediate advantage, under clairvoyant and frozen-exogenous rollouts (%).

Scenario	States	Clairvoyant			Frozen exogenous		
		Agree	Captured	Sacrifice	Agree	Captured	Sacrifice
Full day	201	36	49	60	91	96	7
Evening peak	48	29	42	56	85	97	12
Flash crowd	30	23	65	60	83	99	7
Link failure	30	30	57	57	93	99	3
Deceptive	30	40	52	53	87	92	13
Double failure	30	23	36	73	93	97	7
Overload	21	24	67	38	67	96	5

Table 6.9: Inference cost per decision (process CPU time, one thread, 60 states of a test episode) and training cost (400k transitions, wall time and throughput measured with 3–6 concurrent jobs on 4 cores; the simulator dominates).

Policy	Parameters	Train (min)	Transitions/s	ms / decision	p95 ms
Masked bandit	238,405	45	147	0.112	0.156
MaskablePPO	459,334	49	136	0.335	0.362
MILP-track	–	–	–	31.7	65.7
Greedy	–	–	–	0.0639	0.0758
CSPF	–	–	–	0.0412	0.254
Static SP	–	–	–	0.00342	0.00412
Oracle-1 [†]	–	–	–	359	590

Chapter 7

Engineering, validation and reproduction

7.1 Governance and integrity checks

Every V2 run is wrapped by `AuditedV2Env`, which rejects any action that is illegal under the authoritative pre-action mask, checks that the post-step mask in `info` equals a fresh mask, checks that the twelve reward components sum bit-for-bit to the scalar reward, checks finiteness and flow solver convergence, and re-verifies the protected-class projection of every protected move. Each check aborts the run on failure; consequently the historical “zero failures” counters only certify that runs completed. We therefore audited the mask independently (Section 7.2).

The sixteen files that define the V2 problem are pinned to a signed-off commit and compared byte-for-byte (line-ending canonicalized) before governed training; the ordered candidate-path table is part of every checkpoint’s identity, because V2 reuses V1’s action numbering with different paths.

7.2 Action-mask audit

states	15,480
actions checked	1,068,120
legal te actions	711,848
ppo states checked	15,480
protected legal moves	155,408
transitions	15,480
accepted moves	7,832
frr changes	146
states with failed link	804
states with disconnected demand	144
violation: noop illegal	0
violation: decode mismatch	0
violation: mask validator disagreement	0
violation: masked action accepted	0
violation: masked action side effect	0
violation: legal action rejected	0
violation: legal action wrong path	0
violation: legal action no dwell	0
violation: protected projection violation	0
violation: legal over failed link	0
violation: mask not idempotent	0
violation: independent projection violation	0
violation: independent projection mismatch	0
violation: ppo illegal probability mass	0

7.3 Equivalence and reproduction tests

- The study evaluator (`mplssim/study/episode.py`) reproduces the governed evaluator’s episode summaries exactly (unit test).
- The study trainer reproduces the governed trainer’s network parameters bit for bit after 8,192 transitions for both learners (slow test).
- The $\gamma = 0$ Q-learner is bit-identical to the bandit; the $L = 0$ delayed variant is bit-identical to the frozen environment (unit tests).
- All 21 baseline (policy, scenario) holdout cells of the closed study reproduce to within $3 \cdot 10^{-14}$ on different hardware and operating system.

Chapter 8

Discussion and limitations

8.1 Limitations

Simulator realism. The model is flow-level: delay and loss are analytic functions of utilization, traffic is exogenous and inelastic, the control plane acts instantaneously, fast reroute is free, and telemetry is exact. Learned routing policies trained in fluid models are known not to transfer to packet-level dynamics [16]; our conclusions are about this model. Inelastic traffic in particular removes a natural source of temporal coupling (congestion that suppresses future demand).

Formulation. One trunk may move per interval network-wide and each demand has four fixed candidate paths. These choices create the incremental, stateful character of the problem; a controller that re-optimizes all trunks at once faces the one-shot problem studied by Teal and DOTE instead. We did not vary the number of candidates or the per-interval move limit, because the environment definition is frozen.

Scale and data. One hand-designed 18-router topology, 17 demands, hand-written diurnal profiles and seven scripted test scenarios; no operator traffic matrices or failure traces. No claim of topology generalization is made.

Learning. Five training roots for the primary comparison and two to three for ablations; root-level intervals are wide and are reported. A budget of 400k transitions (about 1,390 simulated days), with one longer run per learner. PPO received a bounded one-factor sensitivity study, the bandit none; neither used graph-structured networks, observation normalization or learning-rate schedules. A discount below the true one can help with limited data [6, 32], so “the myopic learner wins at this budget” does not imply that no sequential structure exists—which is why we measure the structure separately.

Objective. All rankings use one hand-weighted operational reward; we report its components and operational metrics alongside, but a different operator objective could reorder close methods.

References. The clairvoyant oracles see future traffic and only evaluate “one move, then no change”; they bound H -step-greedy reasoning, not the optimal policy. MILP-track optimizes gross maximum utilization only.

Provenance. The closed study’s checkpoints and per-episode holdout data were not archived; its learner results are compared at the aggregate level it committed.

Chapter 9

Process record: failures and design history

9.1 From Environment V1 to V2

The repository’s first version (V1, branch main) trained a single MaskablePPO agent (400k transitions, one seed) on a 586-feature observation and a reward whose terms were mostly clipped. The V2 audit recorded in the code (`mplssim/rl/reward_v2.py`, `mplssim/sim/engine_v2.py`) found that over 13,200 audited V1 intervals the maximum-utilization term was saturated 60.2 % of the time, loss 41.1 % and delay 26.2 %, so a link at 400 % utilization scored like one at 100 %; that congestion and SLA failures were counted twice; and that a flat 0.08 reroute price was repaid by a 0.027 reduction in utilization. The published V1 PPO policy consequently made an accepted TE move in all 3,300 evaluation intervals, 91.7 % of them reversals on `full_day`. V1 also processed link events one tick late (a controller could act on a topology it was told was healthy), reused episode seeds across workers (80 distinct collisions within 30 episodes), admitted two candidate paths that transited provider-edge routers, and loaded every hop of a path with the full offered demand even after upstream drops.

V2 fixed each of these: right-closed event windows, collision-free seeds, role-valid candidates, a carried-flow fixed point, an unsaturated severity-ordered utility, a bounded potential-shaping term and a move cost proportional to what a move disturbs, plus a hard three-interval dwell. Its definition was frozen and signed off (commit `dca533b`) before any V2 learner was trained. These V1 findings are historical (reported in code documentation, not re-executed here).

9.2 The closed V2 study

The V2 study was run as a governed sequence: a seed-42 pilot, a three-root continuity study, and a single untouched final holdout. Its own failures are preserved in its reports: an SB3 seed-propagation bug that invalidated the first PPO run (the wrapper now preserves the governed root and the run fails closed on any other root), two governance repairs needed to admit the preregistered roots, and a CUDA preflight failure before any holdout episode.

9.3 Failures in this study

#	Event	Outcome
1	Full test suite on fresh install	802 passed, 10 skipped, 16 failed; all failures = missing external checkpoints [process]

2	8,192-transition smoke runs of both governed learners (CPU)	ok; 155–157 transitions/s single-process
3	First launch of 6 governed reproduction runs in the main checkout	all 6 aborted at start: governed trainer refuses a dirty checkout (an untracked logs/ directory) [process]. Relaunched in a clean detached worktree at 1457e9b (.worktrees/repro). running
4	Relaunch: roots 42, 314159 (PPO), 42 (bandit) first, 3 in parallel	
5	Sequentiality diagnostic, greedy reference, seeds 4001–4003, $H \leq 24$, clairvoyant	see docs/SEQUENTIALITY_AUDIT.md
6	scripts/study/queue.py crashed on import: module name shadowed stdlib queue (imported by networkx) [process]	renamed to scheduler.py
7	Study-trainer equivalence test (8,192 transitions, both learners)	bit-identical network parameters to the governed trainer
8	Baselines on historical holdout seeds 1001–1005	all 21 (policy, scenario) means reproduce historical values to $\leq 3e-14$ 15,480 states, 1,068,120 checks, 0 violations frozen run queued after #5
9	Mask audit pass 1 (engine-side checks)	
10	Early look at diagnostic (full_day): myopic action agrees with 24-step best in 35 % of states. Added frozen-exogenous counterfactual to separate predictable persistence value from clairvoyant anticipation.	
11	Mask audit pass 2 (+ independent from-scratch projection, + PPO probability mass)	see docs/MASK_VALIDATION.md
12	Diagnostics launched with nice still took 75 % of CPU: kernel autogroup scheduling ignores niceness across sessions [process]	disabled kernel.sched_autogroup_enabled; training throughput restored
13	First reproduction trio finished (bandit r42, PPO r42, PPO r314159); second trio started	evaluated automatically under both protocols
14	Study scheduler had never started (it counted the four low-priority diagnostics as occupying slots); on restart it launched 3 jobs because it matched only commands starting with python and not /usr/bin/python [process]	fixed process matching; scheduler now also skips runs already executing (a restart would otherwise have renamed a live run directory)
15	All 6 reproductions evaluated; E1 bandit r161803, r141421 and PPO r161803 completed	docs/REPRODUCTION_REPORT.md
16	Worker restart killed every background process [process]. Lost: E1 PPO r141421 (608 episodes) and E2 qg05 r42 (672 episodes) mid-training, mask-audit pass 2, partial diagnostics. Persisted: all completed runs and every per-episode output.	interrupted run directories kept as *.interrupted; runs restarted from scratch (study checkpoints hold no replay buffer, so they are not resumable); diagnostics resumed per file; scripts/study/relaunch_background.sh makes relaunch idempotent
17	MILP-track on all 140 test episodes	statistically indistinguishable from the bandit; above PPO [science]
18	Model fidelity (root 42) and deceptive-scenario case study	PPO's actions are worse than no-op over 6 intervals; PPO flips one demand every 3 intervals (hold-down expiry) [science]
19	Return decomposition over all test episodes	on 3 of 4 PPO roots > 50 % of moves are reversals; PPO's network utility is comparable to the bandit's but move/reversal costs are 5–13× larger; shaping contributes ≤ 0.02 per episode [science]

Chapter 10

Conclusion

Bibliography

- [1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, pages 22–31, 2017. URL <https://proceedings.mlr.press/v70/achiam17a.html>.
- [2] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021. URL <https://arxiv.org/abs/2108.13264>. Outstanding Paper Award. arXiv:2108.13264.
- [3] Paul Almasan, José Suárez-Varela, Krzysztof Rusek, Pere Barlet-Ros, and Albert Cabellos-Aparicio. Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case. *Computer Communications*, 196:184–194, 2022. doi: 10.1016/j.comcom.2022.09.029. arXiv:1910.07421.
- [4] Abd AlRhman AlQiam, Yuanjun Yao, Zhaodong Wang, Satyajeet Singh Ahuja, Ying Zhang, Sanjay G. Rao, Bruno Ribeiro, and Mohit Tawarmalani. Transferable neural WAN TE for changing topologies. In *Proceedings of ACM SIGCOMM 2024*, pages 86–102, 2024. doi: 10.1145/3651890.3672237.
- [5] Eitan Altman. *Constrained Markov Decision Processes*. Chapman & Hall/CRC, 1999. doi: 10.1201/9781315140223.
- [6] Ron Amit, Ron Meir, and Kamil Ciosek. Discount factor as a regularizer in reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, 2020. URL <https://arxiv.org/abs/2007.02040>.
- [7] Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphaël Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain Gelly, and Olivier Bachem. What matters for on-policy deep actor-critic methods? A large-scale study. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=nIAxjsniDzg>.
- [8] David Applegate and Edith Cohen. Making intra-domain routing robust to changing and uncertain traffic demands. In *Proceedings of ACM SIGCOMM 2003*, pages 313–324, 2003. doi: 10.1145/863955.863991.
- [9] Pranjal Awasthi, Kush Bhatia, Sreenivas Gollapudi, and Kostas Kollias. Congested bandits: Optimal routing via short-term resets. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *PMLR*, pages 1078–1100, 2022. URL <https://proceedings.mlr.press/v162/awasthi22a.html>.
- [10] D. Awduche, J. Malcolm, J. Agogbua, M. O’Dell, and J. McManus. Requirements for traffic engineering over MPLS. RFC 2702, September 1999. URL <https://www.rfc-editor.org/rfc/rfc2702>.

- [11] D. Awduche, L. Berger, D. Gan, T. Li, V. Srinivasan, and G. Swallow. RSVP-TE: Extensions to RSVP for LSP tunnels. RFC 3209, December 2001. URL <https://www.rfc-editor.org/rfc/rfc3209>.
- [12] Daniel O. Awduche. MPLS and traffic engineering in IP networks. *IEEE Communications Magazine*, 37(12):42–47, 1999. doi: 10.1109/35.809383.
- [13] Guillermo Bernárdez, José Suárez-Varela, Albert López, Bo Wu, Shihan Xiao, Xiangle Cheng, Pere Barlet-Ros, and Albert Cabellos-Aparicio. Is machine learning ready for traffic engineering optimization? In *Proceedings of the 29th IEEE International Conference on Network Protocols (ICNP)*, 2021. doi: 10.1109/ICNP52444.2021.9651930.
- [14] Guillermo Bernárdez, José Suárez-Varela, Albert López, Xiang Shi, Shihan Xiao, Xiangle Cheng, Pere Barlet-Ros, and Albert Cabellos-Aparicio. MAGNETO: A graph neural network-based multi-agent system for traffic engineering. *IEEE Transactions on Cognitive Communications and Networking*, 9(2): 494–506, 2023. doi: 10.1109/TCCN.2023.3235719.
- [15] Alberto Bietti, Alekh Agarwal, and John Langford. A contextual bandit bake-off. *Journal of Machine Learning Research*, 22(133):1–49, 2021. URL <https://jmlr.org/papers/v22/18-863.html>.
- [16] Andreas Boltres, Niklas Freymuth, Patrick Jahnke, Holger Karl, and Gerhard Neumann. Learning sub-second routing optimization in computer networks requires packet-level dynamics. *Transactions on Machine Learning Research*, 2024. URL <https://arxiv.org/abs/2410.10377>. arXiv:2410.10377.
- [17] Justin A. Boyan and Michael L. Littman. Packet routing in dynamically changing networks: A reinforcement learning approach. In *Advances in Neural Information Processing Systems 6 (NIPS 1993)*, pages 671–678, 1993. URL https://papers.nips.cc/paper_files/paper/1993/hash/4ea06fbc83cdd0a06020c35d50e1e89a-Abstract.html.
- [18] Michael Doherty, Robin Matzner, Rasoul Sadeghi, Polina Bayvel, and Alejandra Beghelli. Reinforcement learning for dynamic resource allocation in optical networks: Hype or hope? *Journal of Optical Communications and Networking*, 17(9):D1–D17, 2025. doi: 10.1364/JOCN.559990. arXiv:2502.12804.
- [19] Anwar Elwalid, Cheng Jin, Steven Low, and Indra Widjaja. MATE: MPLS adaptive traffic engineering. In *Proceedings of IEEE INFOCOM 2001*, volume 3, pages 1300–1309, 2001. doi: 10.1109/INFCOM.2001.916625.
- [20] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep RL: A case study on PPO and TRPO. In *International Conference on Learning Representations (ICLR)*, 2020. URL <https://openreview.net/forum?id=r1etN1rtPB>.
- [21] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir. Segment routing architecture. RFC 8402, July 2018. URL <https://www.rfc-editor.org/rfc/rfc8402>.
- [22] Bernard Fortz and Mikkel Thorup. Internet traffic engineering by optimizing OSPF weights. In *Proceedings of IEEE INFOCOM 2000*, volume 2, pages 519–528, 2000. doi: 10.1109/INFCOM.2000.832225.
- [23] Dylan J. Foster and Alexander Rakhlin. Beyond UCB: Optimal and efficient contextual bandits with regression oracles. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, 2020. URL <https://arxiv.org/abs/2002.04926>.

- [24] Javier García and Fernando Fernández. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16(42):1437–1480, 2015. URL <https://jmlr.org/papers/v16/garcia15a.html>.
- [25] Nan Geng, Tian Lan, Vaneet Aggarwal, Yuan Yang, and Mingwei Xu. A multi-agent reinforcement learning perspective on distributed traffic engineering. In *Proceedings of the 28th IEEE International Conference on Network Protocols (ICNP)*, pages 1–11, 2020. doi: 10.1109/ICNP49622.2020.9259413.
- [26] Fei Gui, Songtao Wang, Dan Li, Li Chen, Kaihui Gao, Congcong Min, and Yi Wang. RedTE: Mitigating subsecond traffic bursts with real-time and distributed traffic engineering. In *Proceedings of ACM SIGCOMM 2024*, pages 71–85, 2024. doi: 10.1145/3651890.3672231.
- [27] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11694.
- [28] Chi-Yao Hong, Srikanth Kandula, Ratul Mahajan, Ming Zhang, Vijay Gill, Mohan Nanduri, and Roger Wattenhofer. Achieving high utilization with software-driven WAN. In *Proceedings of ACM SIGCOMM 2013*, pages 15–26, 2013. doi: 10.1145/2486001.2486012.
- [29] Oliver Hope and Eiko Yoneki. GDDR: GNN-based data-driven routing. In *Proceedings of the 41st IEEE International Conference on Distributed Computing Systems (ICDCS)*, pages 517–527, 2021. doi: 10.1109/ICDCS51616.2021.00056.
- [30] Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. In *Proceedings of the International FLAIRS Conference*, volume 35, 2022. doi: 10.32473/flairs.v35i.130584.
- [31] Sushant Jain, Alok Kumar, Subhasree Mandal, Joon Ong, Leon Poutievski, Arjun Singh, Subbaiah Venkata, Jim Wanderer, Junlan Zhou, Min Zhu, Jon Zolla, Urs Hölzle, Stephen Stuart, and Amin Vahdat. B4: Experience with a globally-deployed software defined WAN. In *Proceedings of ACM SIGCOMM 2013*, pages 3–14, 2013. doi: 10.1145/2486001.2486019.
- [32] Nan Jiang, Alex Kulesza, Satinder Singh, and Richard Lewis. The dependence of effective planning horizon on model accuracy. In *Proceedings of the 2015 International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 1181–1189, 2015. URL <https://web.eecs.umich.edu/~baveja/Papers/gamma-AAMAS-final.pdf>. Best Paper Award.
- [33] Srikanth Kandula, Dina Katabi, Bruce Davie, and Anna Charny. Walking the tightrope: Responsive yet stable traffic engineering. In *Proceedings of ACM SIGCOMM 2005*, pages 253–264, 2005. doi: 10.1145/1080091.1080122.
- [34] Praveen Kumar, Yang Yuan, Chris Yu, Nate Foster, Robert Kleinberg, Petr Lapukhov, Chiun Lin Lim, and Robert Soulé. Semi-oblivious traffic engineering: The road not taken. In *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2018. URL <https://www.usenix.org/conference/nsdi18/presentation/kumar>.
- [35] Cassidy Laidlaw, Stuart Russell, and Anca Dragan. Bridging RL theory and practice with the effective horizon. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL <https://arxiv.org/abs/2304.09853>. Oral. arXiv:2304.09853.

- [36] Cassidy Laidlaw, Banghua Zhu, Stuart Russell, and Anca Dragan. The effective horizon explains deep RL performance in stochastic environments. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2312.08369>. Spotlight. arXiv:2312.08369.
- [37] John Langford and Tong Zhang. The epoch-greedy algorithm for multi-armed bandits with side information. In *Advances in Neural Information Processing Systems 20 (NIPS 2007)*, 2007. URL <https://proceedings.neurips.cc/paper/2007/hash/4b04a686b0ad13dce35fa99fa4161c65-Abstract.html>.
- [38] Ximeng Liu, Shizhen Zhao, Yong Cui, and Xinbing Wang. FIGRET: Fine-grained robustness-enhanced traffic engineering. In *Proceedings of ACM SIGCOMM 2024*, pages 117–135, 2024. doi: 10.1145/3651890.3672258.
- [39] Horia Mania, Aurelia Guy, and Benjamin Recht. Simple random search of static linear policies is competitive for reinforcement learning. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018. URL <https://papers.nips.cc/paper/2018/hash/7634ea65a4e6d9041cfd3f7de18e334a-Abstract.html>.
- [40] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533, 2015. doi: 10.1038/nature14236.
- [41] Andrew Y. Ng, Daishi Harada, and Stuart J. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287. Morgan Kaufmann, 1999.
- [42] Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in reinforcement learning. *Journal of Machine Learning Research*, 25(318):1–63, 2024. URL <https://www.jmlr.org/papers/v25/23-0183.html>.
- [43] Xinglong Pei, Penghao Sun, Yuxiang Hu, Dan Li, Bo Chen, and Le Tian. Enabling efficient routing for traffic engineering in SDN with deep reinforcement learning. *Computer Networks*, 241:110220, 2024. doi: 10.1016/j.comnet.2024.110220.
- [44] Yarin Perry, Felipe Vieira Frujeri, Chaim Hoch, Srikanth Kandula, Ishai Menache, Michael Schapira, and Aviv Tamar. DOTE: Rethinking (predictive) WAN traffic engineering. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023. URL <https://www.usenix.org/conference/nsdi23/presentation/perry>. Best Paper Award. arXiv:2303.00735.
- [45] Marek Petrik and Bruno Scherrer. Biasing approximate dynamic programming with a lower discount factor. In *Advances in Neural Information Processing Systems 21 (NIPS 2008)*, pages 1265–1272, 2008. URL https://proceedings.neurips.cc/paper_files/paper/2008/file/08c5433a60135c32e34f46a71175850c-Paper.pdf.
- [46] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL <https://jmlr.org/papers/v22/20-1364.html>.

- [47] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016. URL <https://arxiv.org/abs/1506.02438>.
- [48] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- [49] Giorgio Stampa, Marta Arias, David Sanchez-Charles, Victor Munes-Mulero, and Albert Cabellos. A deep-reinforcement learning approach for software-defined networking routing optimization, 2017. URL <https://arxiv.org/abs/1709.07080>.
- [50] Asaf Valadarsky, Michael Schapira, Dafna Shahaf, and Aviv Tamar. Learning to route. In *Proceedings of the 16th ACM Workshop on Hot Topics in Networks (HotNets-XVI)*, pages 185–191, 2017. doi: 10.1145/3152434.3152441. Extended version: arXiv:1708.03074, “A Machine Learning Approach to Routing”.
- [51] Hao Wang, Haiyong Xie, Lili Qiu, Yang Richard Yang, Yin Zhang, and Albert Greenberg. COPE: Traffic engineering in dynamic networks. In *Proceedings of ACM SIGCOMM 2006*, pages 99–110, 2006. doi: 10.1145/1159913.1159926.
- [52] Ning Wang, Kin Ho, George Pavlou, and Michael Howarth. An overview of routing optimization for Internet traffic engineering. *IEEE Communications Surveys & Tutorials*, 10(1):36–56, 2008. doi: 10.1109/COMST.2008.4483669.
- [53] Zhiying Xu, Francis Y. Yan, Rachee Singh, Justin T. Chiu, Alexander M. Rush, and Minlan Yu. Teal: Learning-accelerated optimization of WAN traffic engineering. In *Proceedings of ACM SIGCOMM 2023*, pages 378–393, 2023. doi: 10.1145/3603269.3604857.
- [54] Zhiyuan Xu, Jian Tang, Jingsong Meng, Weiye Zhang, Yanzhi Wang, Chi Harold Liu, and Dejun Yang. Experience-driven networking: A deep reinforcement learning based approach. In *Proceedings of IEEE INFOCOM 2018*, 2018. doi: 10.1109/INFOCOM.2018.8485853.
- [55] Francis Y. Yan, Hudson Ayers, Chenzhi Zhu, Sadjad Fouladi, James Hong, Keyi Zhang, Philip Levis, and Keith Winstein. Learning in situ: a randomized experiment in video streaming. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 495–511, 2020. URL <https://www.usenix.org/conference/nsdi20/presentation/yan>.
- [56] Junjie Zhang, Minghao Ye, Zehua Guo, Chen-Yu Yen, and H. Jonathan Chao. CFR-RL: Traffic engineering with reinforcement learning in SDN. *IEEE Journal on Selected Areas in Communications*, 38(10):2249–2259, 2020. doi: 10.1109/JSAC.2020.3000371.
- [57] Zihan Zhang, Xiangyang Ji, and Simon S. Du. Is reinforcement learning more difficult than bandits? A near-optimal algorithm escaping the curse of horizon. In *Proceedings of the 34th Conference on Learning Theory (COLT)*, volume 134 of *PMLR*, pages 4528–4531, 2021. URL <https://proceedings.mlr.press/v134/zhang21b.html>.